

Datové typy a pole

datový typ určuje:

- jaký druh hodnoty může proměnná obsahovat
- kolik místa může hodnota zabírat v paměti
- jaké operace s hodnotou můžeme provádět

Rozdělení datových typů

Elementární datové typy

- nesou jednu jednoduchou hodnotu
- hodnota se dál běžně nedělí na menší části
- někdy se jim říká také **primitivní datové typy**

Příklady:

- `int`
- `bool`
- `float`
- `double`

Strukturované datové typy

- skládají se z více hodnot
- mohou obsahovat:
 - elementární typy
 - jiné strukturované typy
- slouží k ukládání složitějších dat

Příklady:

- `array`
- `string`
- `enum`
- `struct`
- `class`

Homogenní a heterogenní struktury

Homogenní struktury

- obsahují prvky stejného typu

- každý prvek má stejný datový typ
- typickým příkladem je pole
- často umožňují přístup pomocí indexu

Příklady:

- pole celých čísel
- pole řetězců
- string jako posloupnost znaků

Příklad:

```
int[] cisla = { 10, 20, 30 };
```

Heterogenní struktury

- obsahují prvky různých typů
- typickým příkladem je třída nebo struktura
- používají se pro popis objektů s více vlastnostmi

Příklad:

```
class Student
{
    public string Jmeno;
    public int Vek;
    public bool Aktivni;
}
```

Pole

Co je pole

- **pole** = strukturovaný datový typ
- obsahuje více prvků stejného typu
- prvky jsou uloženy pod jedním názvem
- k prvkům se přistupuje pomocí indexu
- indexování obvykle začíná od 0

Příklad:

```
int[] cisla = { 10, 20, 30 };
```

Indexy:

```
cisla[0] = 10  
cisla[1] = 20  
cisla[2] = 30
```

Deklarace pole

```
int[] cisla;
```

- říkáme, že proměnná `cisla` bude pole celých čísel

Vytvoření pole

```
cisla = new int[3];
```

- vytvoří pole pro 3 celá čísla
- prvky mají výchozí hodnotu
- u `int` je výchozí hodnota `0`

Deklarace a vytvoření současně

```
int[] cisla = new int[3];
```

Inicializace pole hodnotami

```
int[] cisla = { 10, 20, 30 };
```

Přístup k prvku pole

```
int prvni = cisla[0];
```

- uloží první prvek pole do proměnné `prvni`

Změna prvku pole

```
cisla[1] = 99;
```

- změní druhý prvek pole

Výsledek:

```
{ 10, 99, 30 }
```

Procházení pole cyklem

Cyklus for

```
int[] cisla = { 10, 20, 30 };

for (int i = 0; i < cisla.Length; i++)
{
    Console.WriteLine(cisla[i]);
}
```

- vhodné, když potřebujeme index

Cyklus foreach

```
int[] cisla = { 10, 20, 30 };

foreach (int cislo in cisla)
{
    Console.WriteLine(cislo);
}
```

- vhodné, když nepotřebujeme index
- čitelnější zápis

Vlastnosti pole

Indexování

- prvky se číslují indexem
- první prvek má index 0
- poslední prvek má index `Length - 1`

Příklad:

```
int[] cisla = { 10, 20, 30 };
```

```
první prvek: cisla[0]
poslední prvek: cisla[cisla.Length - 1]
```

Velikost pole

- velikost pole je po vytvoření pevná
- v C# nejde klasické pole jednoduše "nafouknout"
- pokud potřebujeme měnit počet prvků, často se používá `List<T>`

Příklad:

```
int[] cisla = new int[3];
```

- pole má délku 3

Efektivita pole

- čtení prvku podle indexu je rychlé
- zápis prvku podle indexu je rychlý
- obvykle má složitost $O(1)$

Příklad:

```
int x = cisla[2];  
cisla[2] = 50;
```

- program rovnou ví, kde prvek v poli leží

Vkládání a mazání

- vkládání doprostřed pole je pomalejší
- mazání z prostředku pole je pomalejší
- prvky za daným místem se musí posouvat
- obvykle má složitost $O(n)$

Vícerozměrná pole

- pole může mít více rozměrů
- používá se například pro:
 - tabulky
 - matice
 - herní mapy
 - souřadnice

Dvourozměrná pole

- má řádky a sloupce
- v C# se zapisuje pomocí `[,]`
- říká se mu také pravoúhlé pole
- všechny řádky mají stejný počet sloupců

Příklad deklarace:

```
int[,] matice = new int[2, 3];
```

- 2 řádky
- 3 sloupce

Inicializace dvourozměrného pole

```
int[,] matice =  
{  
    { 1, 2, 3 },  
    { 4, 5, 6 }  
};
```

Tabulka:

```
1 2 3  
4 5 6
```

Přístup k prvku dvourozměrného pole

- k prvku se přistupuje pomocí dvou indexů
- první index označuje řádek
- druhý index označuje sloupec

```
int[,] matice =  
{  
    { 1, 2, 3 },  
    { 4, 5, 6 }  
};  
  
int hodnota = matice[0, 1];
```

- vezme hodnotu z řádku 0 a sloupce 1
- výsledek je 2

Změna prvku dvourozměrného pole

```
matice[1, 2] = 99;
```

- změní prvek na řádku 1 a sloupci 2
- původně tam byla hodnota 6
- po změně tam bude hodnota 99

Procházení dvourozměrného pole

```
int[,] matice =
{
    { 1, 2, 3 },
    { 4, 5, 6 }
};

for (int r = 0; r < matice.GetLength(0); r++)
{
    for (int s = 0; s < matice.GetLength(1); s++)
    {
        Console.Write(matice[r, s] + " ");
    }

    Console.WriteLine();
}
```

- `GetLength(0)` vrací počet řádků
- `GetLength(1)` vrací počet sloupců

Jagged pole

- česky: **zubaté pole**
- v C# se zapisuje pomocí `[] []`
- je to pole polí
- každý řádek může mít jinou délku
- hodí se pro nepravidelná data

Rozdíl oproti dvourozměrnému poli

Vlastnost	Dvourozměrné pole <code>[,]</code>	Jagged pole <code>[] []</code>
Struktura	jedna pravoúhlá matice	pole polí
Délka řádků	všechny řádky stejně dlouhé	řádky mohou mít různou délku
Přístup	<code>matice[r, s]</code>	<code>pole[r][s]</code>
Využití	tabulka, matice	nepravidelná data
Paměť	souvislejší blok	více samostatných polí

Příklad jagged pole

```
int[][] zubatePole = new int[3][];

zubatePole[0] = new int[] { 1, 2 };
```

```
zubatePole[1] = new int[] { 3, 4, 5 };  
zubatePole[2] = new int[] { 6 };
```

Tvar:

```
1 2  
3 4 5  
6
```

Přístup k prvku jagged pole

```
int hodnota = zubatePole[1][2];
```

- vezme prvek z řádku 1
- na indexu 2
- výsledek je 5

Procházení jagged pole

```
for (int r = 0; r < zubatePole.Length; r++)  
{  
    for (int s = 0; s < zubatePole[r].Length; s++)  
    {  
        Console.Write(zubatePole[r][s] + " ");  
    }  
  
    Console.WriteLine();  
}
```

- každý řádek může mít jinou délku
- proto se používá `zubatePole[r].Length`

Výčtový typ enum

Co je to enum

- **enum** = výčtový typ
- obsahuje omezenou množinu pojmenovaných hodnot
- používá se tam, kde proměnná může nabývat jen několika stavů
- zvyšuje čitelnost kódu
- omezuje riziko chyb

Příklady použití:

- dny v týdnu

- barvy
- stav objednávky
- role uživatele
- typy účtů

Příklad enumu v C#:

```
enum Den
{
    Pondeli,
    Utery,
    Streda,
    Ctvrtek,
    Patek,
    Sobota,
    Nedele
}
```

Použití:

```
Den dnes = Den.Pondeli;

if (dnes == Den.Pondeli)
{
    Console.WriteLine("Začátek týdne");
}
```

Enum s číselnými hodnotami

- uvnitř je enum obvykle reprezentován číslem
- první hodnota má standardně hodnotu 0
- další se zvyšují o 1
- hodnoty lze nastavit ručně

Příklad:

```
enum StavObjednavky
{
    Vytvorena = 0,
    Zaplacena = 1,
    Odeslana = 2,
    Dorucena = 3,
    Zrusena = 4
}
```

Použití:

```
StavObjednavky stav = StavObjednavky.Zaplacena;
```

Struct a class

Struct

- **struct** = struktura
- hodnotový typ
- používá se pro menší datové celky
- při přiřazení se kopíruje hodnota
- dvě proměnné jsou po kopii nezávislé

Příklad:

```
struct Bod
{
    public int X;
    public int Y;
}
```

Použití:

```
Bod a;
a.X = 10;
a.Y = 20;

Bod b = a;
b.X = 99;

// a.X zůstane 10
```

Class

- **class** = třída
- referenční typ
- používá se pro objekty
- objekt se ukládá na heapu
- proměnná obsahuje referenci na objekt
- při přiřazení se kopíruje reference
- dvě proměnné mohou ukazovat na stejný objekt

Příklad:

```
class Student
{
    public string Jmeno;
    public int Vek;
}
```

Použití:

```
Student a = new Student();
a.Jmeno = "Jan";

Student b = a;
b.Jmeno = "Petr";

// a.Jmeno je také "Petr"
```

Hodnotové a referenční typy

Hodnotové typy

- proměnná obsahuje přímo hodnotu
- při přiřazení se kopíruje hodnota
- změna kopie neovlivní původní hodnotu

Příklady:

- int
- bool
- double
- char
- enum
- struct

Referenční typy

- proměnná obsahuje referenci na objekt v paměti
- při přiřazení se kopíruje reference
- více proměnných může odkazovat na stejný objekt
- změna přes jednu proměnnou se projeví i u druhé

Příklady:

- class
- string
- array

- interface

Poznámka:

- string je referenční typ
- zároveň se chová jako immutable
- po vytvoření se jeho obsah nemění

Praktické ukázky:

1. Vypsání pole

```
int[] cisla = { 5, 10, 15, 20 };

for (int i = 0; i < cisla.Length; i++)
{
    Console.WriteLine(cisla[i]);
}
```

2. Součet prvků pole

```
int[] cisla = { 5, 10, 15, 20 };
int soucet = 0;

for (int i = 0; i < cisla.Length; i++)
{
    soucet += cisla[i];
}

Console.WriteLine(soucet);
```

3. Hledání maxima v poli

```
int[] cisla = { 5, 10, 15, 20 };
int max = cisla[0];

for (int i = 1; i < cisla.Length; i++)
{
    if (cisla[i] > max)
    {
        max = cisla[i];
    }
}

Console.WriteLine(max);
```

4. Vypsání dvourozměrného pole

```
int[,] matice =
{
    { 1, 2, 3 },
    { 4, 5, 6 }
};

for (int r = 0; r < matice.GetLength(0); r++)
{
    for (int s = 0; s < matice.GetLength(1); s++)
    {
        Console.Write(matice[r, s] + " ");
    }

    Console.WriteLine();
}
```

5. Vypsání jagged pole

```
int[][] zubatePole =
{
    new int[] { 1, 2 },
    new int[] { 3, 4, 5 },
    new int[] { 6 }
};

for (int r = 0; r < zubatePole.Length; r++)
{
    for (int s = 0; s < zubatePole[r].Length; s++)
    {
        Console.Write(zubatePole[r][s] + " ");
    }

    Console.WriteLine();
}
```

6. Použití enumu

```
enum StavObjednavky
{
    Vytvorena,
    Zaplacena,
    Odeslana,
    Dorucena
}
```

```
StavObjednavky stav = StavObjednavky.Zaplacena;

if (stav == StavObjednavky.Zaplacena)
{
    Console.WriteLine("Objednávka je zaplacená.");
}
```